

M2Lean: A Certificate-Based Bridge Between Macaulay2 and Lean 4

Andrew Tawfeek*

July 2026

Abstract

Macaulay2 computes; *Lean* certifies. M2Lean connects them through a certificate contract: *Macaulay2*’s answers travel with the *evidence* behind them—cofactors, division data, Buchberger S-pair representations—which *Lean* re-verifies with small checkers backed by kernel-checked soundness theorems, so the resulting theorems rest on no implementation, *Macaulay2*’s or anyone’s. We present a versioned interchange protocol for exact commutative algebra over $\mathbb{Q}$ and prime fields, a *Macaulay2* package that exports computations with their evidence and verifies them without leaving the *Macaulay2* session, and a *Lean* 4 library whose accepted claims become genuine `mathlib` propositions. The Gröbner tier is fully proved: we formalize Buchberger’s criterion in standard-representation form and the degree-reverse-lexicographic monomial order (both absent from `mathlib`), so certified Gröbner bases and negative certificates (non-membership via normal forms) carry kernel-checked soundness. A `by macaulay2` tactic closes ideal-membership goals by consulting a live *Macaulay2* process and kernel-checking its answer. A flagship example proves the twisted cubic cone disjoint from a line over every nontrivial commutative $\mathbb{Q}$ -algebra; a second turns the July 2026 counterexample to the Jacobian conjecture into a kernel-checked theorem; a third certifies that a graph is *not* 3-colourable from a Nullstellensatz infeasibility witness, the counterpart to `mathlib`’s Combinatorial Nullstellensatz; further examples reach into invariant theory, algebraic statistics, Galois theory (a certified `Polynomial.Separable` instance), combinatorial algebraic geometry, and $\mathbb{F}_{101}$. Corrupted certificates are rejected; continuous integration replays everything.

1 Introduction

Somewhere in the literature of commutative algebra, a proof reaches a step that reads: “*a Macaulay2 computation shows that the minimal free resolution has Betti numbers 1, 3, 2.*” Journals accept such sentences routinely, and rightly so: *Macaulay2* is the product of three decades of careful development by a community of mathematicians, its algorithms published, its conventions documented, its behavior exercised daily by researchers who would notice if it were wrong. What the sentence lacks is not credibility but *permanence and portability*. The session behind it is gone — its package versions, options, and monomial-order conventions unrecorded — so the reader a decade later cannot replay it. And no matter how reliable the system that produced it, the result cannot travel into a formal proof, because proof assistants accept nothing on testimony

*Independent researcher. atawfeek.math@gmail.com. The implementation described here was developed with the assistance of Claude (Anthropic). Code and all artifacts: <https://github.com/andrew-tawfeek/M2Lean>.

from *any* software — not from a computer algebra system, not from their own compilers, not from the hardware they run on. That is a different standard of evidence, not a higher one, and it is the standard the celebrated formalizations of the four-color theorem [Gon08], the Kepler conjecture [HAB⁺17], and the Liquid Tensor Experiment [Sch22] were built to meet for their computational steps. M2Lean lets computational commutative algebra meet it routinely and cheaply — without asking anyone to abandon their tools, and without re-litigating *Macaulay2*’s correctness, because what crosses the bridge is mathematical evidence, not an implementation.

The two tools in question serve different masters. A working algebraic geometer who wants to *compute*—a Gröbner basis, a free resolution, a primary decomposition—reaches for a computer algebra system such as *Macaulay2* [GS24]. A mathematician who wants to *certify*—to produce a machine-checked proof—reaches for a proof assistant such as *Lean* 4 [dMU21] with its mathematical library *mathlib* [The20]. These ought to be two halves of one workflow; in practice they are disconnected, because *Lean*’s algebra library, while remarkably broad, contains almost none of the executable machinery of computational commutative algebra, and *Macaulay2*’s answers carry no evidence a proof assistant could accept.

M2Lean is a bridge between the two systems designed around a division of labor rather than a translation:

<i>Macaulay2</i>	the <i>computational laboratory</i> : discovers examples, computes bases, resolutions, invariants, decompositions
<i>Lean</i>	the <i>logical authority</i> : fixes the mathematical meaning of those computations and certifies conclusions

The goal is not to translate *Macaulay2* source code into *Lean*, nor to verify *Macaulay2*’s implementation. It is to create a shared mathematical language and a bidirectional protocol through which a computation can cross from one system to the other *without losing its meaning, its assumptions, or its checkability*.

What a certificate is. The mechanism throughout is the *certificate*: data that lets a computational claim be checked by hand, without redoing the work that found it. Deciding that $x^3 + y^3$ lies in the ideal $(x + y, x - y)$ takes a Gröbner computation, but the engine can hand back the *cofactors* that witness the answer—the **red** multipliers are the ones *Macaulay2* had to find; $x + y$ and $x - y$ are just the given generators:

$$x^3 + y^3 = \frac{1}{2}(x^2 + y^2)(x + y) + \frac{1}{2}(x^2 - y^2)(x - y),$$

and now anyone—a referee, a student, or a proof kernel—confirms membership just by multiplying out and comparing, with no ideal theory at all. Two properties make this worth building a bridge around. A *wrong* certificate cannot pass the check, so accepting one never requires trusting the system that produced it; and the check is small and self-contained, so the fact it establishes is permanent and portable in a way a vanished *Macaulay2* session is not. Every result in this paper travels with evidence of this shape—cofactors for membership, division quotients for S-polynomial reductions, change-of-basis data for Gröbner bases—and *Lean* accepts nothing until it has re-checked the evidence for itself.

Finding is hard; checking is easy. The entire design rests on one asymmetry. Deciding ideal membership is EXPSPACE-complete in general [MM82]; Gröbner engines represent decades of

algorithmic engineering precisely because the *search* is expensive. But the witness the search produces — cofactors expressing an element in the generators, division quotients realizing an S-polynomial reduction — can be verified by checking finitely many polynomial identities, a task a patient student could do by hand. M2Lean asks *Macaulay2* only for what its algorithms already possess internally (witnesses) and asks *Lean* only for what a proof kernel is built to do (check them). Neither system is asked to become the other. This is why the bridge is cheap: certified answers in this paper cost milliseconds to re-verify and kilobytes to store (§13), while remaining checkable by any *Lean* installation, forever, with every assumption pinned down.

What arrives is more than what was sent. A certificate crosses the bridge as arithmetic and arrives as geometry. In the flagship example (§10), *Macaulay2*’s witness is a two-line polynomial identity, $1 = (-1)(xz - y^2) + (-y - 1)(y - 1) + x \cdot z$; the *Lean* theorem it yields says that the cone over the twisted cubic and a line have no common point over *every* nontrivial commutative $\mathbb{Q}$ -algebra — a quantification *Macaulay2* cannot even express. In the Galois example (Appendix B), a two-term Bézout identity arrives as `Polynomial.Separable`, a named concept of the mathematical library, ready to feed into field theory. The bridge is not a printer: *Lean* adds generality and meaning that *Macaulay2* does not have, and *Macaulay2* supplies discoveries that *Lean* could not have found.

The skeptic’s contract. M2Lean adopts the classical skeptical stance toward computer algebra [HT98]: architecturally, *Macaulay2* plays the role of an *untrusted oracle*. The term is one of art, not a verdict on software quality. Formal proof draws its trust boundary around a tiny kernel and places *everything* else outside it—its own elaborator, its compilers, the hardware—and *Macaulay2*, whose development and testing practices are as rigorous as any in mathematical software, simply sits on the same side of that line as everything else in the world. Concretely: a result is never accepted because *Macaulay2* printed `true`; it is accepted when *Macaulay2* supplies *evidence*—cofactors expressing an element in terms of generators, change-of-basis data, division quotients for S-polynomials—and a small *Lean* checker verifies that evidence. For the core checkers we prove soundness theorems in *Lean*: if the checker accepts, a genuine `mathlib` proposition holds. The discipline spares everyone an argument: a defect anywhere in the bridge code, the serialization, or even *Macaulay2* itself could only cause spurious *rejection*, never a false theorem—so the bridge never needs to adjudicate *Macaulay2*’s correctness at all.

Contributions. This paper describes the design and a complete working vertical slice of the bridge, the *M2Lean Bridge Core*:

1. **A versioned interchange protocol** (§5) for exact commutative algebra over $\mathbb{Q}$ and prime fields: polynomial rings with normatively specified monomial orders, ideals, matrices, graded free modules, and seven kinds of *claims with evidence*, including negative certificates. Canonical forms are strict enough that equality of encodings is structural, and consumers reject rather than repair.
2. **A *Macaulay2* bridge package** (§6) that exports objects, produces certificates from public *Macaulay2* interfaces (including S-pair evidence from an explicit division algorithm), and closes the loop in-session: `verifyWithLean` D spawns the *Lean* verifier on the current document and displays colored per-claim verdicts without leaving *Macaulay2*.

3. **A *Lean* 4 library** (§7) with an executable sparse model over any coefficient field (instantiated at $\mathbb{Q}$ and $\mathbb{ZMod}\ p$), a strict protocol validator, certificate checkers, and machine-checked soundness theorems producing genuine **mathlib** propositions.
4. **A formalized Gröbner tier** (§8): Buchberger’s criterion in standard-representation form (`buchberger_criterion`) and the degree-reverse-lexicographic order as a **mathlib** `MonomialOrder` (`degRevLex`), both new formalizations and natural **mathlib** upstream candidates, plus the bridge lemmas connecting them to the executable checker. Gröbner and non-membership certificates over `GRevLex` are **proved**, not merely **checked**.
5. **A by *macaulay2* tactic** (§9): ideal-membership goals are discharged by consulting a live *Macaulay2* subprocess and kernel-checking the returned certificate at elaboration time.
6. **Flagship theorems and cross-area examples** (§10, §11, §12, Appendix B): certified disjointness of the twisted cubic cone and a line over every nontrivial commutative $\mathbb{Q}$ -algebra; a kernel-checked refutation of the Jacobian conjecture as stated, from the July 2026 dimension-three counterexample; a certified `¬SimpleGraph.Colorable` from a Nullstellensatz infeasibility witness—the nonexistence counterpart to **mathlib**’s Combinatorial Nullstellensatz; a kernel-checked `Polynomial.Separable` for $X^4 + 1$ over $\mathbb{Q}$; Gröbner, membership, and non-membership over $\mathbb{F}_{101}$; and more — with an axiom audit showing no additions to *Lean*’s standard three axioms.
7. **Adversarial testing and continuous integration** (§13): mutated real certificates and malformed documents are rejected with failure-class diagnostics; a GitHub Actions pipeline regenerates every *Macaulay2* document, rebuilds the *Lean* library (exercising the tactic against a live *Macaulay2*), and replays all 19 checks and the axiom audit on every push.

How to read this paper. Readers coming from *Macaulay2* may prefer §2, §6, the flagship walk-through (§10), and the condensed examples of Appendix B; none of these require reading *Lean*. Readers coming from *Lean* will find the mathematical content in §7–§9, especially the Gröbner formalization (§8). Readers inclined to trust neither system should start with the trust model (§4) and the adversarial evaluation (§13).

Everything is open source; the repository contains the protocol specification, both implementations, all fixtures, and scripts that reproduce every claim in this paper [Taw26].

2 What this buys you

If you use *Macaulay2*. Your computations become citable. A session that today ends with a printed Betti table can end with one extra call, `verifyWithLean D`, and produce a certificate that any *Lean* installation — a referee’s, a reader’s, one running twenty years from now — can re-check, with the coefficient field, monomial order, grading, and package versions pinned down machine-readably. You never write a line of *Lean*. The discipline pays even before verification: making certificates exportable required exact answers to questions that need none inside a single system but must be pinned down for results to travel (which `GRevLex` tie-break convention is in force; what the division algorithm guarantees about its quotients), and the shared adversarial fixtures will catch any future drift on either side. And the statuses are

honest: a computation is reported as *computed*, *checked*, or *proved*, never a conflation of the three.

If you use Lean. Thirty years of computational commutative algebra become available to your proofs without anyone reimplementing them. You do not need a verified Gröbner engine to use a Gröbner basis: you need a witness and a checker, and the checker’s soundness theorem makes the result a genuine *mathlib* proposition — here, span membership, ideal equality, non-membership, and vanishing compositions, consumed by ordinary theorems with no new axioms and no *sorry*. The bridge is also a formalization engine rather than a bypass: Buchberger’s criterion and the degree-reverse-lexicographic order entered *Lean* because the checker demanded them (§8), and both are stated against *mathlib*’s own *MonomialOrder* theory for upstreaming. At the ergonomic end, *macaulay2* (§9) makes the round trip a single tactic call.

3 Related work

Skeptical combination of proof assistants and computer algebra goes back to Harrison and Théry’s HOL–Maple bridge [HT98], which introduced the pattern we follow: let the CAS find an object that is expensive to discover but cheap to check (a factorization, an antiderivative), and re-verify it in the prover. Delahaye and Mayero connected Coq to Maple for field simplification [DM05], and Kaliszyk and Wiedijk prototyped a full CAS on top of HOL Light [KW07]. Within *Lean*, the *linear_combination* tactic [Gol22] checks exactly the kind of cofactor certificate our membership checker consumes, and *polyrith* [Bha22] obtains those cofactors by calling an external Gröbner engine—a special case of the M2Lean pattern, built directly into a tactic. M2Lean generalizes this from single tactic calls to a versioned, provenance-carrying protocol spanning objects (rings, ideals, matrices, complexes) and claim families, with an explicitly stratified trust model.

On the formalization side, Buchberger’s algorithm was verified in Coq by Théry [Thé01] and Gröbner theory is developed in Isabelle/HOL [IM16]; *mathlib* had no comparable development, a gap this work partially closes with a proof of Buchberger’s criterion and the degree-reverse-lexicographic order (§8), stated against *mathlib*’s *MonomialOrder* theory for upstreaming. CoqEAL [CDM13] refines abstract algebraic structures to efficient executable ones inside the prover; M2Lean is complementary, keeping the heavy computation outside the prover and importing only evidence. Certificate-based cooperation is also standard practice in SMT proof reconstruction [BBFF20] and in sums-of-squares certificates for nonlinear arithmetic [Har07]; M2Lean can be read as bringing that discipline to computational commutative algebra, where the natural certificates are cofactors, syzygies, and division data [CLO15, BW93].

4 Architecture and trust model

4.1 Four layers

layer	realization
interchange representation	<code>protocol/SPEC.md</code> , JSON schema, fixtures
semantics	<i>Lean</i> : <code>sparse model</code> $\rightarrow$ <code>MvPolynomial (Fin n) ℚ</code>
claims and certificates	<i>Lean</i> checkers + soundness theorems
user-facing integrations	<i>Macaulay2</i> package; <code>m2lean-check</code> CLI

A document travels left to right: *Macaulay2* exports objects and claims, *Lean* parses and *validates* (arities, canonical scalars, dependency order, dimension checks—rejecting, never re-pairing), then runs one checker per claim and emits a verification report; claims whose checkers carry soundness theorems can additionally be consumed by *Lean* theorem files through the kernel (Figure 1 traces one such trip end to end). Parsing is not proof: the semantics layer is what turns byte-level checks into mathematics.

Throughout the paper, code listings are color-coded by which side of the trust boundary owns the artifact: **red** for *Macaulay2* (computation and evidence generation, outside the kernel-checked base), **amber** for protocol documents (the data crossing the boundary), and **blue** for *Lean* (validation, checking, and theorems).

4.2 Trusted and untrusted

“Trusted” and “untrusted” name sides of an architectural boundary—what a false theorem could depend on versus what it cannot—not degrees of confidence in anyone’s code. The trusted base consists of the *Lean* kernel, the definitions giving protocol objects mathematical meaning (the sparse model and its interpretation), the statements of the soundness theorems, and **mathlib** itself. Outside the boundary, by design: all of *Macaulay2* (engine, interpreter, our package), all serialization and transport, every field of every document including provenance, and generated *Lean* syntax prior to kernel checking. Placing *Macaulay2* outside is what makes the bridge *light* for *Macaulay2*: nothing about its internals needs to be audited, frozen, or re-implemented for the theorems to stand. Checkers may not use `native_decide`; certificate checks in theorem files run through the kernel (`decide +kernel`).

4.3 Assurance levels

Every accepted claim carries one of four levels, and the distinction is preserved end to end—in the checker code, in the CLI report, and in which claims example theorems are permitted to consume:

<code>transported</code>	object parsed and validated; nothing checked
<code>reproduced</code>	computation re-run and matched; no certificate
<code>checked</code>	certificate verified by an executable checker whose mathematical soundness is cited but not yet formalized
<code>proved</code>	as <code>checked</code> , plus a kernel-checked soundness theorem: acceptance <i>implies</i> the mathlib proposition

In the current implementation, polynomial identity, ideal membership, span inclusion, chain-complex composition, Gröbner bases, and non-membership are **proved** (the latter two over GRevLex, *Macaulay2*’s default order; their Lex instances remain `checked` until the Lex agreement lemmas are formalized); graded complexes are `checked`. Claims are checked in the coefficient field their ring declares: $\mathbb{Q}$ or, for `PrimeField p` documents, $\mathbb{ZMod}\ p$. We regard surfacing this distinction as a design contribution rather than a confession: systems that blur “the checker ran” and “the theorem holds” invite exactly the silent failures a bridge like this exists to eliminate.

5 The interchange protocol

The protocol (currently version 0.2.0, the string consumers refuse to mismatch) deliberately covers a small, exactly specified fragment: coefficients in $\mathbb{Q}$ (reduced fractions of decimal strings; JSON numbers are never used for mathematical data) or $\mathbb{F}_p$ (p verified prime by the consumer); multivariate polynomial rings with positional variable identity and one of two normatively defined monomial orders (**Lex** and **GRevLex**, *Macaulay2*’s default); sparse polynomials in canonical form (nonzero reduced coefficients, exact arity, strictly decreasing terms); ideals; matrices; and graded free modules $\bigoplus_j R(-d_j)$.

Canonicity is strict enough that equality of encoded polynomials is structural equality, which is the base of every checker, and it is enforced, not assumed: a document whose terms are misordered for the *declared* order is rejected. This addresses a classical source of silent cross-system corruption—graded reverse lexicographic order alone admits several incompatible textbook conventions—by making the definitions normative in the specification and testing both implementations against shared adversarial fixtures ordered differently under **Lex** and **GRevLex**.

A *claim* is a proposition about previously introduced objects plus evidence addressed to a specific checker:

claim kind	proposition	evidence
PolynomialIdentity	$lhs = rhs$	none (normalization)
IdealMembership	$f \in (g_1, \dots, g_k)$	cofactors c_i with $f = \sum c_i g_i$
SpanInclusion	$I \subseteq J$	a cofactor row per generator
GroebnerBasis	G is a GB of I	span witnesses both ways + S-pair standard representations
NonMembership	$f \notin I$	division data vs. a certified GB: $f = \sum q_k b_k + r$, $r \neq 0$ reduced
ChainComplex	$d_i \circ d_{i+1} = 0$	none (matrix arithmetic)
GradedComplex	the above, graded	none (homogeneity checks)

Documents also carry provenance—producer versions, algorithm names, determinism flags—which is displayed for reproducibility but never influences acceptance. The verifier answers with a report document whose failure messages distinguish parse, structural, and verification failures.

The most interesting evidence format is the Gröbner one. For every pair $i < j$ of basis elements whose leading monomials are not coprime, the producer must supply quotients $q_1, \dots, q_r$ such that

$$S(b_i, b_j) = \sum_k q_k b_k, \quad \text{lm}(q_k b_k) \preceq \text{lm}(S(b_i, b_j)) \text{ whenever } q_k \neq 0,$$

i.e. a *standard representation*, for *every* pair (coprime pairs may not be omitted: this keeps the soundness proof within the standard-representation criterion, without Buchberger’s first criterion). By Buchberger’s criterion in standard-representation form [BW93, Thm. 5.64] [Buc70, CLO15], verifying these identities and bounds (plus the two span inclusions relating G to the input generators) establishes that G is a Gröbner basis. The point of this format is that the checker never runs Buchberger’s algorithm: it only checks polynomial identities and monomial comparisons, both of which reduce to the structural equality of canonical forms. Since the criterion itself is now formalized (§8), this implication is no longer a citation but a kernel-checked theorem.

Negative certificates ride on positive ones: a **NonMembership** claim references a **GroebnerBasis** claim in the same document and supplies division data with a nonzero remainder no term of

which is divisible by a basis leading monomial. Its soundness genuinely *requires* the Gröbner property — a reduced nonzero remainder refutes membership only against a true Gröbner basis — which is why this claim kind exists at the **proved** level only on the strength of the Buchberger formalization.

6 The Macaulay2 bridge package

The package `m2/M2Lean.m2` exports objects and generates certificates using public *Macaulay2* interfaces. From the user’s point of view, encoding mathematics for the bridge is ordinary *Macaulay2*: one builds rings and ideals as usual, then attaches claims to a document and writes it out. A complete session certifying a membership looks like this:

```
loadPackage "M2Lean";
R = QQ[x,y];
I = ideal(x+y, x-y); -- the mathematics, as usual
D = newM2LeanDocument "session-1";
membershipClaim(D, "mem1", x^3 + y^3, I); -- computes the cofactors
writeM2LeanDocument(D, "session-1.json"); -- crosses the boundary
```

The loop closes without leaving the session:

```
verifyWithLean D
-- m2lean-check report for document session-1:
-- mem1: accepted [proved] element lies in the ideal
```

`verifyWithLean` writes the document to a temporary file, spawns the Lean verifier as a subprocess, parses the report, and displays colored per-claim verdicts (green **proved**, amber **checked**, red rejections), returning the parsed report for programmatic use.

`membershipClaim` silently does the interesting work: it divides the element against the ideal’s Gröbner basis, extracts cofactors, re-checks the identity, and serializes everything—generators, element, evidence, monomial order, provenance—in the canonical forms of §5. Because everything the package produces is re-verified, it may freely use any convenient *Macaulay2* machinery: membership cofactors come from matrix division `f // gens I` against *Macaulay2*’s internal Gröbner engine, and basis conversion witnesses likewise. The one place where honesty about *how* a result was obtained matters is the S-pair evidence: *Macaulay2*’s engine does not expose its reduction traces, so the package re-derives the quotients with an explicit textbook division algorithm (twenty lines of *Macaulay2* code) whose loop invariant guarantees the standard-representation bound that the protocol demands. This is a small instance of a phenomenon we expect to recur: *certificate export forces implicit algorithmic guarantees to become explicit*, and occasionally forces recomputation in a shape that carries its own correctness evidence.

One subtlety deserves record. *Macaulay2* keeps polynomials sorted in the ring’s monomial order, so canonical export is free—but only after the bridge verifies that *Macaulay2*’s order conventions agree with the protocol’s normative definitions. Our test suite pins this down empirically (e.g. $x + y^2z$ must serialize as $[x, y^2z]$ under **Lex** and $[y^2z, x]$ under **GRevLex**), and the shared fixtures would catch any drift on either side.

7 The Lean library

7.1 An executable model and its interpretation

The computational heart is a deliberately small sparse model, parameterized over the coefficient field (instantiated at $\mathbb{Q}$ for `RationalField` documents and `ZMod p` for `PrimeField` documents, after the verifier confirms primality and synthesizes the field instance):

```
structure STerm (α : Type*) where
  coeff : α
  exps : List Nat -- positional; entries beyond the end are 0
abbrev SPoly (α : Type*) := List (STerm α)
```

with executable normalization (sort by a fixed total comparator, then merge equal exponent vectors and drop zeros, with trailing-zero trimming so that paddings of the same monomial coincide), ring operations on raw term lists, and the two protocol monomial orders. All recursion is structural, for a reason that becomes visible later: well-founded recursion does not reduce definitionally, and these functions must run *inside the kernel*.

The semantics is a function `toMv n : SPoly → MvPolynomial (Fin n) ℚ` interpreting a term list as a sum of monomials, together with a lemma suite proved once and for all: `toMv` commutes with concatenation, term-wise product, permutation, merging, and normalization. The payoff is the soundness of the two executable predicates every checker is built from:

```
theorem polyEq_sound (h : polyEq p q = true) : toMv n p = toMv n q
theorem polyIsZero_sound (h : polyIsZero p = true) : toMv n p = 0
```

7.2 Checkers and soundness theorems

Checkers are Boolean functions on plain sparse data—independent of the protocol AST, so the soundness theorems apply to them directly. Representative statements (all `sorry`-free, all checked by the kernel):

```
theorem checkMembership_sound
  (h : checkMembership f gs cs = true) :
  toMv n f ∈ Ideal.span {p | p ∈ gs.map (toMv n)}

theorem checkSpanInclusion_sound
  (h : checkSpanInclusion src tgt rows = true) :
  spanOf n src ≤ spanOf n tgt

theorem checkComplexPair_sound
  (h : checkComplexPair r m c A B = true) :
  toMatrix n r m A * toMatrix n m c B = 0

theorem no_common_zero
  (h : checkMembership onePoly gs cs = true)
  {K : Type*} [CommRing K] [Nontrivial K] [Algebra ℚ K]
  (pt : Fin 4 → K)
  (hv : ∀ g ∈ gs, aeval pt (toMv n g) = 0) : False
```

The last theorem is the abstract half of the flagship: a certificate that $1 = \sum c_i g_i$ makes the span the unit ideal, and the unit ideal has no common zero in any nontrivial commutative

$\mathbb{Q}$ -algebra—the certificate refutes points valued in every field extension at once, which is exactly the content usually delivered by the Nullstellensatz but obtained here by pure algebra from an explicit witness.

The Gröbner checker `checkGroebner` verifies S-pair coverage, each standard representation with its leading-monomial side conditions, and both span inclusions. The implication “criterion verified $\Rightarrow$ Gröbner basis” is not cited here but proved: §8 describes the formalization that puts these claims at the `proved` level — for every certificate ever produced, with no change to *Macaulay2* or the protocol.

7.3 Kernel-checked certificate consumption

The CLI accepts documents at runtime, but example *theorems* embed certificate data as literals (generated deterministically from the JSON by a checked-in script) and verify them by kernel reduction:

```
theorem certificate_checks :
  checkMembership onePoly gens cofactors = true := by decide +kernel
```

Two implementation notes may save others time. First, any well-founded recursion in a checker silently blocks such kernel evaluation; our merge step is a structural `foldr` for exactly this reason. Second, `Rat` arithmetic does not reduce in the elaborator’s evaluator but does in the kernel (which has fast arbitrary-precision arithmetic including `Nat.gcd`), so `decide +kernel` rather than plain `decide` is required.

8 The formalized Gröbner tier

The Gröbner checker is not merely executable: its acceptance *implies* the Gröbner property by a kernel-checked proof. Three formalizations were required, none previously available in `mathlib`.

Buchberger’s criterion. We prove the standard-representation form of Buchberger’s criterion [Buc70, BW93] over an arbitrary `mathlib` `MonomialOrder` and field:

```
theorem buchberger_criterion
  (hb : ∀ i, b i ≠ 0)
  (hS : ∀ i j, HasStdRep m b (sPoly m (b i) (b j)))
  (hf : f ∈ Ideal.span (Set.range b)) (hf0 : f ≠ 0) :
  ∃ i, m.degree (b i) ≤ m.degree f
```

The proof is the classical one, mechanized: pick a representation of f minimizing (by well-foundedness of the order) the largest product degree δ ; if δ exceeds $\deg f$, the top-degree layer cancels and is rewritten through S-polynomial standard representations — a cancellation lemma proved by strong induction on the size of the layer — yielding a representation with smaller δ . Verified Buchberger developments exist in Coq [Thé01] and Isabelle [IM16]; this appears to be the first in *Lean*, and it is deliberately stated against `mathlib`’s own `MonomialOrder/degree` theory so it can be upstreamed next to `MonomialOrder.div`.

The degree-reverse-lexicographic order. `mathlib` provides `lex` and `degLex`, but not *Macaulay2*’s default order. We construct `MonomialOrder.degRevLex` as a type synonym ordered by total degree, ties broken by the *order dual* of `mathlib`’s colexicographic comparison. One

subtlety deserves record: unlike `degLex`, whose tie-break order is itself well-founded, plain reverse-lexicographic comparison is *not* $(1 \succ x_0 \succ x_0^2 \succ \dots)$, so well-foundedness cannot be inherited componentwise. It holds anyway because degrees are non-increasing along a descending chain and, over finitely many variables, only finitely many exponent vectors exist below a given degree — which is why `degRevLex` requires `[Finite  $\sigma$ ]`, a hypothesis `degLex` does not need.

The bridge, and checkers that check what the proof needs. Connecting the executable checker to the abstract criterion requires agreement lemmas — the byte-level comparator on exponent lists computes exactly the `degRevLex` relation on interpreted monomials — and a *lead bridge*: the head of the sorted normalization is the abstract leading term. The latter would need a formal theory of sorting-with-merging; instead, the checker was refined to *certify* the facts the proof needs. A new `leadOK` check asserts, decidably, that the head strictly dominates every other term; soundness then consumes the check rather than a meta-theorem about the sorting algorithm. This “check what the proof needs” pattern kept the trusted development hundreds of lines smaller, at zero cost on honest certificates. The final statements:

```
theorem checkGroebner_sound
  (h : checkGroebner n .grevlex gens basis bc gc sps = true) :
  spanOf n gens = spanOf n basis ∧
  ∀ f ∈ spanOf n basis, f ≠ 0 →
    ∃ b ∈ basis, (grevlexOrder n).degree (toMv n b) ≤
      (grevlexOrder n).degree f

theorem checkNonMembership_sound
  (hgb : checkGroebner n .grevlex gens basis bc gc sps = true)
  (hnm : checkNonMembership n .grevlex basis f quotes r = true) :
  toMv n f ∉ spanOf n basis
```

Both hold over any coefficient field, so they apply verbatim to $\mathbb{F}_p$ documents. The second is the promised negative certificate: division data with a nonzero reduced remainder refutes membership — *only* against a proved Gröbner basis, which is why it ships in the same section. Certificates over `Lex` rings remain at `checked` until the (entirely analogous) `Lex` agreement lemmas are formalized; the assurance level is computed per order and displayed per claim.

9 The by macaulay2 tactic

The architecture compresses into a single tactic call:

```
def f : SPoly ℚ := [⟨1, [3, 0]⟩, ⟨1, [0, 3]⟩] -- x3 + y3
def gs : List (SPoly ℚ) :=
  [[⟨1, [1, 0]⟩, ⟨1, [0, 1]⟩], [⟨1, [1, 0]⟩, ⟨-1, [0, 1]⟩]]

example : toMv 2 f ∈ spanOf 2 gs := by macaulay2
```

The tactic reflects the goal’s sparse data, writes a *Macaulay2* session requesting a membership certificate, runs `M2 -script` as a subprocess, parses the returned protocol document with the `M2Lean` parser, embeds the cofactors as literals, and closes the goal with `checkMembership_sound` applied to a kernel-checked run of the certificate checker. The trust story is unchanged: the answer enters the proof as data, not authority — if anything in the round trip misfires (the subprocess, the parse, the transcription), the kernel check fails and the tactic reports an error; it

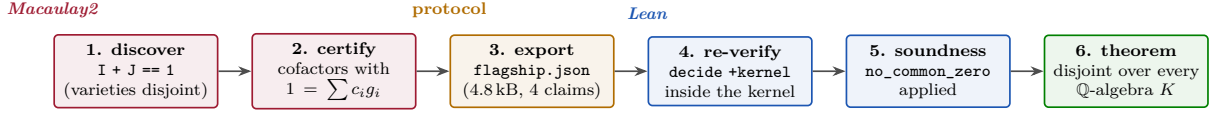


Figure 1: The flagship path. *Macaulay2* finds the fact and the evidence; the protocol carries them as data; *Lean* re-checks the evidence in its kernel and turns it into a theorem quantified over every nontrivial commutative $\mathbb{Q}$ -algebra.

can never prove a falsehood. The demo file above is built in continuous integration, so every CI run exercises a live *Lean*-to-*Macaulay2* round trip at elaboration time. This prototype handles ground membership goals; goal reflection for symbolic polynomials (via `ring_nf`-style normalization, as in `polyrith` [Bha22]) is the natural next step.

10 The flagship example

The example is chosen to be mathematically recognizable, small enough for continuous integration, and structured so that both systems are essential (cf. [Eis95]). Let $R = \mathbb{Q}[x, y, z, w]$ and let

$$I = (xz - y^2, xw - yz, yw - z^2)$$

be the ideal of 2×2 minors of $\begin{pmatrix} x & y & z \\ y & z & w \end{pmatrix}$, cutting out the cone over the twisted cubic curve. Let $L = V(y - 1, z, w)$ be the line $\{(t, 1, 0, 0)\}$. Figure 1 traces the artifacts through the pipeline; the remainder of this section walks the same path in detail.

Step 1: discovery (*Macaulay2*). The mathematics is encoded in *Macaulay2* exactly as a researcher would write it by hand:

```

R = QQ[x,y,z,w];
I = minors(2, matrix{{x,y,z},{y,z,w}}); -- the twisted cubic cone
J = ideal(y - 1, z, w); -- the line {(t,1,0,0)}
assert(I + J == 1); -- discovery: disjoint!

```

This is the kind of fact a researcher finds in seconds and would previously have had to re-derive by hand to use in a formal development.

Step 2: certificate (*Macaulay2*). Five claims are attached to a document and exported:

```

D = newM2LeanDocument "flagship";
unitIdealClaim(D, "unit1", I + J); -- 1 = sum c_i g_i
gbClaim(D, "gb1", I); -- Buchberger evidence
C = res comodule I;
nonMembershipClaim(D, "nm1", x, I); -- x does NOT vanish on C
gradedComplexClaim(D, "res1", {C.dd_1, C.dd_2});
chainComplexClaim(D, "cx1", {C.dd_1, C.dd_2});
writeM2LeanDocument(D, "flagship.json");

```

The membership certificate for 1 carries cofactors—in this instance

$$1 = (-1) \cdot (xz - y^2) + (-y - 1) \cdot (y - 1) + x \cdot z,$$

a two-line identity that is trivial to check and encapsulates the entire computation. It also exports a Gröbner certificate for I (three S-pairs with standard representations) and the minimal graded free resolution of R/I , the Eagon–Northcott complex [EN62]

$$0 \rightarrow R(-3)^2 \xrightarrow{d_2} R(-2)^3 \xrightarrow{d_1} R \rightarrow R/I \rightarrow 0,$$

as a graded-complex claim. The full document is 4.8 kB of JSON.

Step 3: verification (*Lean*). `m2lean-check` accepts all five claims and reports their assurance levels — note that the Gröbner and non-membership claims are now **proved** (§8):

claim	verdict	level
unit1 (membership of 1 in $I+J$)	accepted	proved
gb1 (Gröbner basis of I)	accepted	proved
nm1 ($x \notin I$)	accepted	proved
res1 (graded complex, Betti data 1, 3, 2)	accepted	checked
cx1 ($d_1 d_2 = 0$)	accepted	proved

Step 4: mathematics (*Lean*). The example file re-checks the certificate in the kernel and applies the soundness theorems, yielding (verbatim from the repository):

```

theorem twisted_cubic_cone_misses_line
  {K : Type*} [CommRing K] [Nontrivial K] [Algebra ℚ K]
  (p : Fin 4 → K)
  (h1 : aeval p (X 0 * X 2 - X 1 ^ 2) = 0)
  (h2 : aeval p (X 0 * X 3 - X 1 * X 2) = 0)
  (h3 : aeval p (X 1 * X 3 - X 2 ^ 2) = 0)
  (h4 : aeval p (X 1 - 1) = 0)
  (h5 : aeval p (X 2) = 0)
  (h6 : aeval p (X 3) = 0) : False

theorem resolution_composes_to_zero :
  toMatrix 4 1 3 resD1 * toMatrix 4 3 2 resD2 = 0

```

The first statement quantifies over every nontrivial commutative $\mathbb{Q}$ -algebra: no point of any base change lies on both varieties. *Macaulay2* computed; the certificate crossed the boundary; *Lean* supplied semantics and generality. Neither system alone produces this artifact: *Macaulay2* has no notion of “for all $\mathbb{Q}$ -algebras K ,” and *Lean* would not have found the cofactors. An axiom audit confirms that these theorems depend on exactly *Lean*’s three standard axioms (`propext`, `Classical.choice`, `Quot.sound`)—no sorry, no project-specific axioms, no `native_decide`.

11 A second flagship: the Jacobian conjecture

The flagship of §10 was chosen for pedagogy; this one was chosen by events. The *Jacobian conjecture* (O.-H. Keller, 1939 [Kel39]) asserts that a polynomial map $F : \mathbb{C}^n \rightarrow \mathbb{C}^n$ whose Jacobian determinant is a nonzero constant has a polynomial inverse. It had been open for every $n \geq 2$ for eighty-seven years and is among the best-known problems in the subject. On 19 July 2026 Levent Alpöge announced an explicit counterexample in dimension three, found with the assistance of Anthropic’s Claude Fable 5 model after Akhil Mathew proposed

the search [Alp26, Spa26]. Because the witness is a handful of polynomial evaluations it was independently checked by many mathematicians within a day [Spa26, Sec26], and T. Tao related it to the classical degree reductions of Bass–Connell–Wright [BCW82] and Yagzhev [Yag80].

This is precisely the situation M2Lean is built for, and it exercises the skeptic’s contract of §1 at full strength: the bridge does not care who—or what—produced the map, nor how. The assertion that it *is* a counterexample splits into two exact, independently re-verifiable facts: a nonzero-constant Jacobian determinant (a polynomial identity) and a failure of injectivity (a finite collision of rational points). Provenance never enters; what *Lean* certifies rests on its kernel alone. The map is

$$F(x, y, z) = \begin{pmatrix} (1 + xy)^3 z + y^2(1 + xy)(4 + 3xy), \\ y + 3x(1 + xy)^2 z + 3xy^2(4 + 3xy), \\ 2x - 3x^2 y - x^3 z. \end{pmatrix}$$

Step 1: the hypothesis holds (*Macaulay2*). *Macaulay2* forms the Jacobian and watches its determinant collapse to a constant—the conjecture’s hypothesis, satisfied:

```
R = QQ[x,y,z];
F = {(1+x*y)^3*z + y^2*(1+x*y)*(4+3*x*y),
      y + 3*x*(1+x*y)^2*z + 3*x*y^2*(4+3*x*y),
      2*x - 3*x^2*y - x^3*z};
Jac = matrix table(3,3,(i,j) -> diff({x,y,z}#j, F#i));
assert(det Jac == -2); -- nonzero constant Jacobian
polynomialIdentityClaim(D, "detJac", det Jac, -2_R);
```

Step 2: the conclusion fails (*Macaulay2*). The three distinct points $P_0 = (0, 0, -\frac{1}{4})$, $P_1 = (1, -\frac{3}{2}, \frac{13}{2})$, and $P_2 = (-1, \frac{3}{2}, \frac{13}{2})$ all map to $(-\frac{1}{4}, 0, 0)$, so F is not injective and has no inverse of any kind. Each component value $F_i(P) = v_i$ is exported as a membership in the point’s maximal ideal $\mathfrak{m}_P = (x-a, y-b, z-c)$: the cofactor identity $F_i - v_i = A(x-a) + B(y-b) + C(z-c)$, evaluated at P , kills the ideal part and forces $F_i(P) = v_i$. Nine such membership claims (three points $\times$ three coordinates) together with the determinant identity form a 12.4 kB document; `m2lean-check` reports all ten claims **proved**.

Step 3: the theorem (*Lean*). *Lean* re-checks the nine certificates in its kernel, proves the Jacobian determinant constant directly over `mathlib`’s `MvPolynomial` via `pderiv`, and assembles the refutation (verbatim from the repository):

```
theorem jacF_det : jacF.det = -2 -- the hypothesis, in mathlib

theorem F_not_injective : -- the conclusion: F is not injective
  ¬Function.Injective (fun p : Fin 3 → ℚ => fun i => aeval p (F i))

theorem jacobian_conjecture_false : ¬JacobianConjecture 3
```

Here `JacobianConjecture n` is the injective formulation over $\mathbb{Q}$ —if the Jacobian determinant is a nonzero constant, the induced self-map of $\mathbb{Q}^n$ is injective—so refuting even injectivity is the strongest possible counterexample. The non-injectivity half is *Macaulay2*’s discovery, and the asymmetry of §1 is stark: *Lean* was never asked to *search* for the colliding points—the hard

part—and need only *check* that they collide, which its kernel does in a handful of evaluations. *Macaulay2*’s nine membership certificates are the evidence that crosses the bridge: `m2lean-check` accepts them, and the theorem file re-checks them in the kernel (`certificate_checks`) and promotes them to genuine ideal-membership propositions through the membership soundness theorem (`collision_certified`, §7). As with the flagship, an axiom audit confirms `jacobian_conjecture_false` and its lemmas depend on exactly *Lean*’s three standard axioms—no `sorry`, no project-specific axioms, no `native_decide`. The plane case $n = 2$ remains open; the counterexample extends to every $n \geq 3$ by adjoining identity coordinates. We take no view on the mathematics beyond what we re-derived: the point is that a claimed counterexample of real consequence became, through the bridge, a kernel-checked *Lean* theorem in an afternoon, its correctness resting on nothing but *Lean*’s kernel.

12 Certifying nonexistence: graph colouring

The examples so far certify that something *holds*. A certificate can equally prove that something is *impossible*, and graph colouring is the sharpest illustration. Fix a graph G and three colours. Following De Loera–Lee–Malkin–Margulies [DLLMM08], encode a colouring algebraically: one variable x_v per vertex, a vertex relation $x_v^3 - 1$ (“the colour is a cube root of unity”), and for each edge an edge relation $x_u^2 + x_u x_v + x_v^2 = (x_u^3 - x_v^3)/(x_u - x_v)$ (“adjacent colours differ”). A proper 3-colouring is exactly a common zero of these polynomials, so

$$G \text{ is 3-colourable} \iff 1 \notin \text{col}(G),$$

and a Nullstellensatz certificate $1 = \sum_i c_i g_i$ is a proof of *non-colourability*.

A clean contrast with mathlib. `mathlib` already contains Alon’s *Combinatorial Nullstellensatz* [Alo99] (`Mathlib.Combinatorics.Nullstellensatz`), the top-coefficient nonvanishing criterion that is the standard algebraic tool for proving that colourings (and many other combinatorial objects) *exist*; it has no counterpart for proving that they do not. M2Lean supplies exactly that missing direction and lands on the same named predicate: from *Macaulay2*’s certificate, *Lean* concludes $\neg \text{SimpleGraph.Colorable } G \ 3$. The two are complementary—existence by non-vanishing, nonexistence by ideal membership.

Encoding correctness (*Lean*). This bridge does more than re-check arithmetic: it must connect the ideal membership to the combinatorial predicate. *Lean* proves, once and generically, that a proper 3-colouring c furnishes the common zero $x_v = \zeta^{c(v)}$ (with ζ a primitive cube root of unity in $\mathbb{C}$) of every generator—a vertex relation vanishes because $(\zeta^{c(v)})^3 = 1$, an edge relation because $\zeta^{3c(u)} = \zeta^{3c(v)} = 1$ while $\zeta^{c(u)} \neq \zeta^{c(v)}$. Feeding this zero to `no_common_zero` against *Macaulay2*’s kernel-checked certificate is a contradiction, yielding (verbatim):

```
| theorem wheel5_not_three_colorable : ¬W.Colorable 3
```

for the odd wheel W_5 , with an axiom audit confirming *Lean*’s three standard axioms and no `sorry`.

Finding is hard; checking stays easy—quantitatively. Deciding 3-colourability is NP-complete, and the Nullstellensatz certificate grows with the instance; yet every certificate is

checked by the same trivial polynomial arithmetic. The table makes the asymmetry concrete, from the clique K_4 to the *Grötzsch graph*—the Mycielskian of the 5-cycle [Myc55], the smallest triangle-free graph that is not 3-colourable, where no clique forces the fourth colour so the obstruction is genuinely global:

graph	vertices	edges	certificate (kB)	check (ms)
K_4 (clique)	4	6	4.5	31
W_5 (odd wheel)	6	10	8.0	32
Grötzsch (triangle-free)	11	20	112	772

The witness grows 25×; verification stays under a second. The Grötzsch certificate is verified by `m2lean-check (examples/coloring/)`; W_5 additionally carries the full kernel-checked *Lean* theorem above.

13 Adversarial evaluation

The acceptance criterion that matters most for a bridge is not that honest certificates pass but that dishonest ones fail. The test suite (`scripts/run-checks.sh`) exercises both directions over fourteen valid documents (eleven *Macaulay2*-generated, including the Jacobian counterexample of §11, the colouring certificates of §12, and the cross-area examples of Appendix B) and eight corrupted or malformed ones; the corrupted certificates are obtained by *mutating real Macaulay2 output*, not by constructing strawmen:

document	defect	outcome
14 valid documents	—	all accepted
<code>corrupt-membership</code>	wrong cofactor	rejected (verification)
<code>corrupt-gb-quotient</code>	S-pair quotient scaled by 2	rejected (verification)
<code>corrupt-gb-missing-spair</code>	required S-pair deleted	rejected (verification)
<code>corrupt-complex</code>	differential entry altered	rejected (verification)
<code>corrupt-nonmembership</code>	remainder zeroed out	rejected (verification)
<code>noncanonical-unsorted</code>	terms misordered for ring order	rejected (structural)
<code>structure-bad-arity</code>	exponent arity mismatch	rejected (structural)
<code>parse-truncated</code>	invalid JSON	rejected (parse)

Cost. The finding-is-hard/checking-is-easy asymmetry of §1 is visible in measurements (one laptop core, median of three runs):

document	claims	size (kB)	verification (ms)
<code>flagship</code> (unit ideal, GB, non-mem., resolution)	5	5.3	33
<code>jacobian</code> (Jacobian det. + 9 collision certificates)	10	12.4	34
<code>coloring</code> (K_4 Nullstellensatz witness)	1	4.5	32
<code>polynomial</code> (identity, membership, span ×2)	4	2.5	31
<code>primefield</code> (GB, membership, non-mem. over $\mathbb{F}_{101}$)	3	2.3	34
<code>galois</code> (Φ_8 separability witness)	1	0.9	31

Verification wall time is dominated by process startup — the checking itself is sub-millisecond at this scale — and generating *all* documents with *Macaulay2* takes under a second (`scripts/bench.sh`). Kernel-level re-verification is the expensive tier, and still cheap:

within a warm build, elaborating the Galois theorem file (certificate check plus the transfer to `Polynomial.Separable`) takes 2.9s, and the flagship file (two kernel certificate checks plus the interpretation lemmas) 5.6s; and the Jacobian file (nine kernel certificate checks, the determinant computation over `MvPolynomial`, and the soundness transfer) 4.3s. At this scale, certificates are the same order of magnitude as the answers they certify; how they grow is a limitation discussed in §15. The whole pipeline runs in continuous integration on every push: a GitHub Actions job installs *Macaulay2* and *Lean*, rebuilds the library (including the `by macaulay2` demo, which spawns a live *Macaulay2* process during elaboration), regenerates all documents, replays the 22 accept/reject checks, and greps the axiom audit.

14 What each system gains

For *Lean*. Immediately: a disciplined way to import serious computation. The pattern “*Macaulay2* finds the witness, a small checker consumes it” extends well beyond membership—elimination witnesses, Hilbert-series data, syzygies, radical membership via Rabinowitsch, regular sequences—and each new certificate kind is a checker plus a soundness theorem, not a verified reimplementation of a CAS. Less obviously: the checkers themselves generate a formalization agenda ordered by computational relevance. Our Gröbner checker is an exact, executable statement of what remains to be proved (Buchberger’s criterion for standard representations over the two protocol orders); formalizing against a fixed executable target is a better-posed task than formalizing a textbook chapter.

For *Macaulay2*. The bridge is also a mirror. Exporting certificates forced precise answers to questions that are usually implicit: exactly which monomial order convention is in force, what the division algorithm guarantees about its quotients, which results are deterministic. We expect the deeper influence to run through specifications: “`certify gb I`” gives *Macaulay2* users a machine-checkable notion of what `gb` promises, distinct from what the implementation happens to do—and a user-visible difference between “computed” and “certified” creates the incentive for algorithms to expose the evidence they already possess internally (change matrices and reduction traces that engines routinely compute and discard).

15 Limitations and roadmap

The current fragment is exactly what the trust model can carry today, and its edges are documented rather than blurred. `ChainComplex` certifies a complex, not exactness—the specification is explicit that $d_i d_{i+1} = 0$ establishes only a complex, and exactness certificates (via certified syzygy claims, or rank and depth data in the style of Buchsbaum–Eisenbud) are the designed next step of the homological tier. Graded claims and Lex-order Gröbner claims are **checked**, not **proved**: the graded semantics and the Lex agreement lemmas (a smaller sibling of the `GRevLex` bridge of §8) remain to be formalized. Certificates are eagerly embedded rather than streamed, which will not survive contact with large computations without a compact encoding and reflective checkers. The `by macaulay2` tactic handles ground membership goals; reflecting symbolic goals is open.

What about *your* computation? The examples in this paper are deliberately CI-sized; a reader with a research-scale computation should calibrate. What scales today: membership-style

certificates, whose size tracks the cofactors (often comparable to the Gröbner basis itself), and everything on the *Macaulay2* side, which is just ordinary *Macaulay2* plus serialization. What does not yet: Gröbner certificates carry one standard representation per S-pair, so for large bases the evidence can dwarf the answer — the classical certificate-explosion risk — and the current JSON encoding and eagerly-embedded kernel checks would both need the planned compact encoding, streaming, and reflective checking before a hundred-generator ideal is comfortable. Nothing in the *architecture* obstructs this: the protocol is encoding-independent by design, and the soundness theorems are already stated against data, not files.

The near-term roadmap, in order of leverage: upstream `buchberger_criterion` and `degRevLex` to `mathlib`; formalize the Lex agreement lemmas and the graded semantics; add exactness certificates; reinstate the coprime-pair optimization by formalizing Buchberger’s first criterion; and extend the tactic to symbolic goals. The long-term test of the project is whether conjectures, computations, certificates, and theorems can move between the two ecosystems while retaining precise meaning and explicit trust—so that a computation performed in 2026 can still be replayed, and its theorem still trusted, by systems that have long since changed.

Acknowledgements

This project’s design and implementation were carried out with the assistance of Claude (Anthropic). We thank the Macaulay2 and *Lean/mathlib* communities, whose public interfaces and libraries made a two-system bridge feasible as a vertical slice.

References

- [Alo99] Noga Alon. Combinatorial Nullstellensatz. *Combinatorics, Probability and Computing*, 8(1–2):7–29, 1999.
- [Alp26] Levent Alpöge. An explicit counterexample to the Jacobian conjecture in dimension three. Announcement, `@__alpöge__`; reported in [Spa26], 2026. 19 July 2026.
- [BBFF20] Haniel Barbosa, Jasmin Christian Blanchette, Mathias Fleury, and Pascal Fontaine. Scalable fine-grained proofs for formula processing. In *Journal of Automated Reasoning*, volume 64, pages 485–510, 2020.
- [BCW82] Hyman Bass, Edwin H. Connell, and David Wright. The Jacobian conjecture: reduction of degree and formal expansion of the inverse. *Bulletin of the American Mathematical Society*, 7(2):287–330, 1982.
- [Bha22] Dhruv Bhatia. `polyrith`: a Lean tactic closing polynomial arithmetic goals via an external computer algebra system. `mathlib4`, https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Polyrith.html, 2022.
- [Buc70] Bruno Buchberger. Ein algorithmisches Kriterium für die Lösbarkeit eines algebraischen Gleichungssystems. *Aequationes Mathematicae*, 4:374–383, 1970.
- [BW93] Thomas Becker and Volker Weispfenning. *Gröbner Bases: A Computational Approach to Commutative Algebra*, volume 141 of *Graduate Texts in Mathematics*. Springer, 1993.

- [CDM13] Cyril Cohen, Maxime Dénès, and Anders Mörtberg. Refinements for free! In *Certified Programs and Proofs (CPP 2013)*, volume 8307 of *Lecture Notes in Computer Science*, pages 147–162. Springer, 2013.
- [CLO15] David A. Cox, John Little, and Donal O’Shea. *Ideals, Varieties, and Algorithms*. Undergraduate Texts in Mathematics. Springer, 4th edition, 2015.
- [DLLMM08] Jesús A. De Loera, Jon Lee, Peter N. Malkin, and Susan Margulies. Hilbert’s Nullstellensatz and an algorithm for proving combinatorial infeasibility. In *Proceedings of the 21st International Symposium on Symbolic and Algebraic Computation (ISSAC 2008)*, pages 197–206. ACM, 2008.
- [DM05] David Delahaye and Micaela Mayero. Dealing with algebraic expressions over a field in Coq using Maple. In *Journal of Symbolic Computation*, volume 39, pages 569–592, 2005.
- [dMU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [DS98] Persi Diaconis and Bernd Sturmfels. Algebraic algorithms for sampling from conditional distributions. *Annals of Statistics*, 26(1):363–397, 1998.
- [Eis95] David Eisenbud. *Commutative Algebra with a View Toward Algebraic Geometry*, volume 150 of *Graduate Texts in Mathematics*. Springer, 1995.
- [EN62] J. A. Eagon and D. G. Northcott. Ideals defined by matrices and a certain complex associated with them. *Proceedings of the Royal Society of London. Series A*, 269:188–204, 1962.
- [Gol22] Abby J. Goldberg. `linear_combination`: certifying ring equalities from explicit coefficients. mathlib4, https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/LinearCombination.html, 2022.
- [Gon08] Georges Gonthier. Formal proof—the four-color theorem. *Notices of the American Mathematical Society*, 55(11):1382–1393, 2008.
- [GS24] Daniel R. Grayson and Michael E. Stillman. Macaulay2, a software system for research in algebraic geometry. Available at <http://www2.macaulay2.com>, 2024.
- [HAB⁺17] Thomas Hales, Mark Adams, Gertrud Bauer, et al. A formal proof of the Kepler conjecture. *Forum of Mathematics, Pi*, 5:e2, 2017.
- [Har07] John Harrison. Verifying nonlinear real formulas via sums of squares. *Theorem Proving in Higher Order Logics (TPHOLs 2007)*, LNCS 4732, pages 102–118, 2007.
- [HT98] John Harrison and Laurent Théry. A skeptic’s approach to combining HOL and Maple. *Journal of Automated Reasoning*, 21(3):279–294, 1998.
- [IM16] Fabian Immler and Alexander Maletzky. Gröbner bases theory. *Archive of Formal Proofs*, 2016. https://isa-afp.org/entries/Groebner_Bases.html, formal proof development.

- [Kel39] Ott-Heinrich Keller. Ganze Cremona-transformationen. *Monatshefte für Mathematik und Physik*, 47:299–306, 1939.
- [KW07] Cezary Kaliszyk and Freek Wiedijk. Certified computer algebra on top of an interactive theorem prover. In *Towards Mechanized Mathematical Assistants (Calculus/MKM 2007)*, volume 4573 of *Lecture Notes in Computer Science*, pages 94–105. Springer, 2007.
- [MM82] Ernst W. Mayr and Albert R. Meyer. The complexity of the word problems for commutative semigroups and polynomial ideals. *Advances in Mathematics*, 46(3):305–329, 1982.
- [Myc55] Jan Mycielski. Sur le coloriage des graphes. *Colloquium Mathematicum*, 3(2):161–162, 1955.
- [Sch22] Peter Scholze. Liquid tensor experiment. *Experimental Mathematics*, 31(2):349–354, 2022.
- [Sec26] Secret Blogging Seminar. The new counterexample to the Jacobian conjecture. <https://sbseminar.wordpress.com/2026/07/20/the-new-counterexample-to-the-jacobian-conjecture/>, 2026. 20 July 2026.
- [Spa26] Matthew Sparkes. AI’s solution to 87-year-old riddle takes mathematicians by surprise. *New Scientist*, 2026. 20 July 2026.
- [Taw26] Andrew Tawfeek. M2lean: repository and artifact. <https://github.com/andrew-tawfeek/M2Lean>, 2026.
- [Thé01] Laurent Théry. A machine-checked implementation of Buchberger’s algorithm. *Journal of Automated Reasoning*, 26(2):107–137, 2001.
- [The20] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.
- [Yag80] A. V. Yagzhev. Keller’s problem. *Siberian Mathematical Journal*, 21(5):747–754, 1980.

A A complete protocol document

The valid fixture `membership.json`, abridged only in whitespace: the claim certifies $2x \in (x + y, x - y)$ in $\mathbb{Q}[x, y]$ with cofactors $(1, 1)$.

```
{ "m2leanVersion": "0.1.0",
  "documentId": "fixture-membership",
  "objects": [
    { "id": "QQ", "kind": "RationalField" },
    { "id": "R", "kind": "PolynomialRing", "coefficients": "QQ",
      "variables": ["x", "y"], "monomialOrder": { "kind": "GRevLex" } },
    { "id": "I", "kind": "Ideal", "ring": "R", "generators": [
```

```

{ "terms": [
  { "coefficient": {"num":"1","den":"1"}, "exponents": [1,0] },
  { "coefficient": {"num":"1","den":"1"}, "exponents": [0,1] } ] },
{ "terms": [
  { "coefficient": {"num":"1","den":"1"}, "exponents": [1,0] },
  { "coefficient": {"num":"-1","den":"1"}, "exponents": [0,1] } ] } ] },
"claims": [
{ "id": "c1", "kind": "IdealMembership", "ideal": "I",
  "element": { "terms": [
    { "coefficient": {"num":"2","den":"1"}, "exponents": [1,0] } ] },
  "evidence": { "cofactors": [
    { "terms": [ { "coefficient": {"num":"1","den":"1"},
      "exponents": [0,0] } ] },
    { "terms": [ { "coefficient": {"num":"1","den":"1"},
      "exponents": [0,0] } ] } ] } ] },
"provenance": { "producer": "handwritten", "producerVersion": "n/a",
  "packageVersion": "0.1.0", "deterministic": true } } ]

```

The corresponding verification report:

```

{ "m2leanVersion": "0.1.0",
  "reportFor": "fixture-membership",
  "results": [
    { "claim": "c1", "status": "accepted", "assurance": "proved",
      "message": "element lies in the ideal" } ],
  "provenance": { "producer": "m2lean-check (Lean 4)",
    "packageVersion": "0.1.0" } }

```

B Five more crossings

Each example below (six, spanning six areas) is one complete trip across the bridge, condensed to its core: the *Macaulay2* discovery (red), the certificate that crosses the protocol boundary, and what *Lean* verifies (blue). All five documents are generated by `examples/appendix/appendix.m2` and verified by `m2lean-check` in the test suite; the certificate identities displayed are quoted from the actual documents. Each **proved**-level claim could be embedded and kernel-checked as a *Lean* theorem exactly as in §10; we keep these five at the CLI level for brevity.

B.1 Invariant theory: Newton’s identity

Power sums are polynomials in elementary symmetric functions; the degree-3 instance is a `PolynomialIdentity` claim, the base case of the certificate family.

```

R = QQ[x,y,z];
polynomialIdentityClaim(D, "newton3",
  x^3+y^3+z^3, e1^3 - 3*e1*e2 + 3*e3);

```

$$p_3 = e_1^3 - 3e_1e_2 + 3e_3.$$

Lean: ✓ **accepted** [**proved**] — both raw sides normalize to the same canonical form, so their interpretations in `MvPolynomial (Fin 3) ℚ` are equal (`checkIdentity_sound`).

B.2 Toric ideals and algebraic statistics: Markov moves

For 2×3 contingency tables, the toric ideal of the independence model contains three 2×2 minors, but Markov chains often use only the *adjacent* moves $m_1 = p_{11}p_{22} - p_{12}p_{21}$ and $m_2 = p_{12}p_{23} - p_{13}p_{22}$ [DS98]. The long move becomes reachable whenever the middle entry is positive:

```
S = QQ[p11,p12,p13,p21,p22,p23];
membershipClaim(D, "markov",
  p22*(p11*p23 - p13*p21), ideal(m1, m2));
```

$$p_{22}(p_{11}p_{23} - p_{13}p_{21}) = p_{23}m_1 + p_{21}m_2.$$

Lean: ✓ accepted [proved] — the element lies in $\text{span}(m_1, m_2)$ (`checkMembership_sound`); the displayed cofactors (p_{23}, p_{21}) are *Macaulay2*'s.

B.3 Galois theory: separability of Φ_8

A polynomial is separable iff it is coprime to its derivative; the Bézout witness is a unit-ideal certificate.

```
T = QQ[t]; f = t^4 + 1;
unitIdealClaim(D, "separable", ideal(f, diff(t, f)));
```

$$1 = 1 \cdot (t^4 + 1) + \left(-\frac{t}{4}\right) \cdot 4t^3.$$

Lean: ✓ accepted [proved] — this crossing is fully promoted: the certificate is embedded and kernel-checked (`Examples/Galois.lean`), the Bézout identity is transferred to univariate polynomials, and the result is `mathlib`'s own separability notion, `phi8_separable`: the polynomial $X^4 + 1$ is separable in $\mathbb{Q}[X]$, in `mathlib`'s own sense — Φ_8 has no repeated root in any extension of $\mathbb{Q}$, i.e. $\mathbb{Q}(\zeta_8)/\mathbb{Q}$ is étale.

B.4 Combinatorics: K_4 is not 3-colorable

Encoding colors as cube roots of unity, a graph G is 3-colorable iff the system $\{x_i^3 - 1\} \cup \{x_i^2 + x_i x_j + x_j^2 : ij \in E(G)\}$ has a solution; infeasibility is certified by a Nullstellensatz witness [DLLMM08].

```
G = QQ[x1,x2,x3,x4];
U = ideal(vertexPolys | edgePolys); -- 4 + 6 generators
assert(U == 1); -- discovery
unitIdealClaim(D, "k4not3col", U);
```

The ten cofactors are genuinely computed objects—the one attached to $x_1^2 + x_1 x_2 + x_2^2$ is

$$\frac{2}{9}x_3^2x_4^2 + \frac{2}{9}x_3x_4^3 - \frac{4}{9}x_4^4 - \frac{1}{3}x_3 + \frac{1}{3}x_4,$$

not something one guesses by hand.

Lean: ✓ accepted [proved] — $1 \in U$, hence the system has no solution in any nontrivial commutative $\mathbb{Q}$ -algebra; by the classical encoding argument, K_4 admits no proper 3-coloring.

B.5 Combinatorial algebraic geometry: a Stanley–Reisner resolution

The Stanley–Reisner ideal of the 4-cycle is the complete intersection $I = (ac, bd)$; its Koszul resolution has Betti numbers 1, 2, 1.

```
Q = QQ[a,b,c,d]; Isr = ideal(a*c, b*d);
C = res_comodule Isr;
gradedComplexClaim(D, "koszul", {C.dd_1, C.dd_2});
chainComplexClaim(D, "koszulcx", {C.dd_1, C.dd_2});
```

$$0 \rightarrow R(-4) \xrightarrow{\begin{pmatrix} -bd \\ ac \end{pmatrix}} R(-2)^2 \xrightarrow{(ac \ bd)} R \rightarrow R/I \rightarrow 0.$$

Lean: ✓ accepted [proved / checked] — $d_1 d_2 = 0$ as a `mathlib Matrix` identity (`checkComplexPair_sound, proved`); homogeneity of every entry for the displayed twists—hence the Betti table 1, 2, 1—is verified at level `checked`.

B.6 Finite fields: the same machinery over $\mathbb{F}_{101}$

The coefficient field is generic: the same checkers and the same soundness theorems run over `ZMod p`.

```
F = ZZ/101[x,y]; Ip = ideal(x^2 + y^2, x*y - 1);
gbClaim(D, "gbp", Ip);
membershipClaim(D, "memp", x^3 + x*y^2, Ip);
nonMembershipClaim(D, "nmp", x, Ip); -- x is NOT in Ip
```

Lean: ✓ accepted [proved] — all three claims — the Gröbner basis, the membership, and the negative certificate — verify over `ZMod 101` at the `proved` level; the Buchberger soundness theorem is field-generic, so finite-field certificates inherit it for free.